

PROJET FIL ROUGE · CDA 2026

Amoura

Application web de restaurant · Réservation & fidélité

JALON 4 – AVRIL 2026

Conception de l'application & Architecture

Formation CDA · Symfony 7 + React + MariaDB + Docker

Amoura

1. Diagrammes de cas d'utilisation

Le diagramme recense toutes les interactions entre les acteurs et le système. Trois acteurs : le **Client** (connecté), l'**Admin** (restaurateur) et le **Service Email** (Mailjet). La relation <<include>> entre Réserver et Envoyer email représente l'envoi automatique de confirmation.

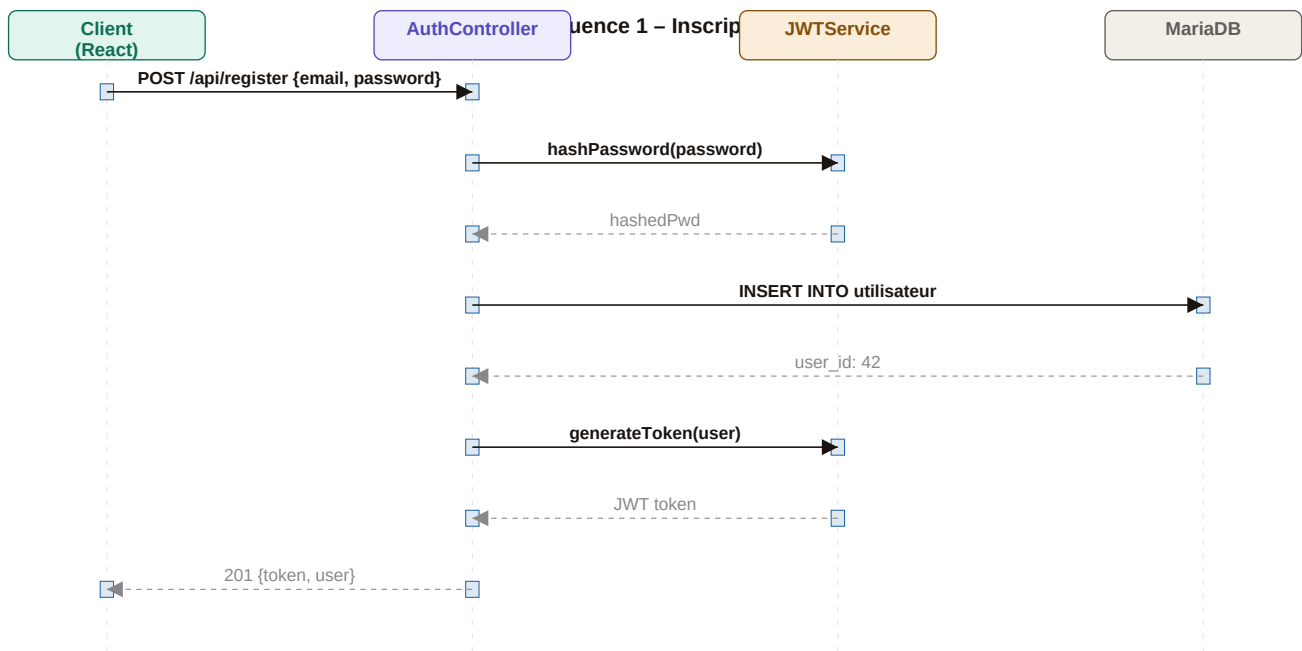


2. Diagrammes de séquence

J'ai détaillé trois scénarios couvrant les flux principaux : l'inscription, la réservation et l'attribution des points fidélité.

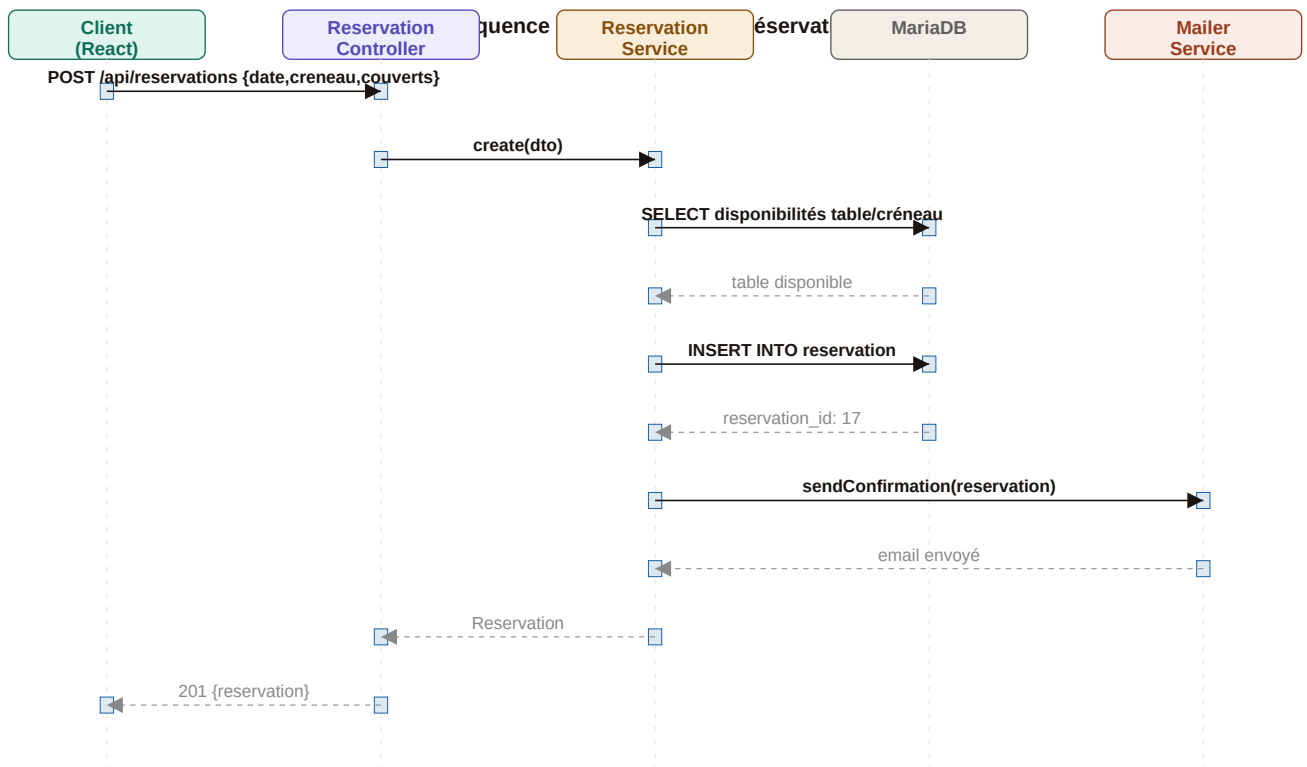
2.1 Inscription d'un nouveau client

L'utilisateur soumet le formulaire React. AuthController hache le mot de passe via JWTService, persiste l'utilisateur en base et retourne un token JWT.



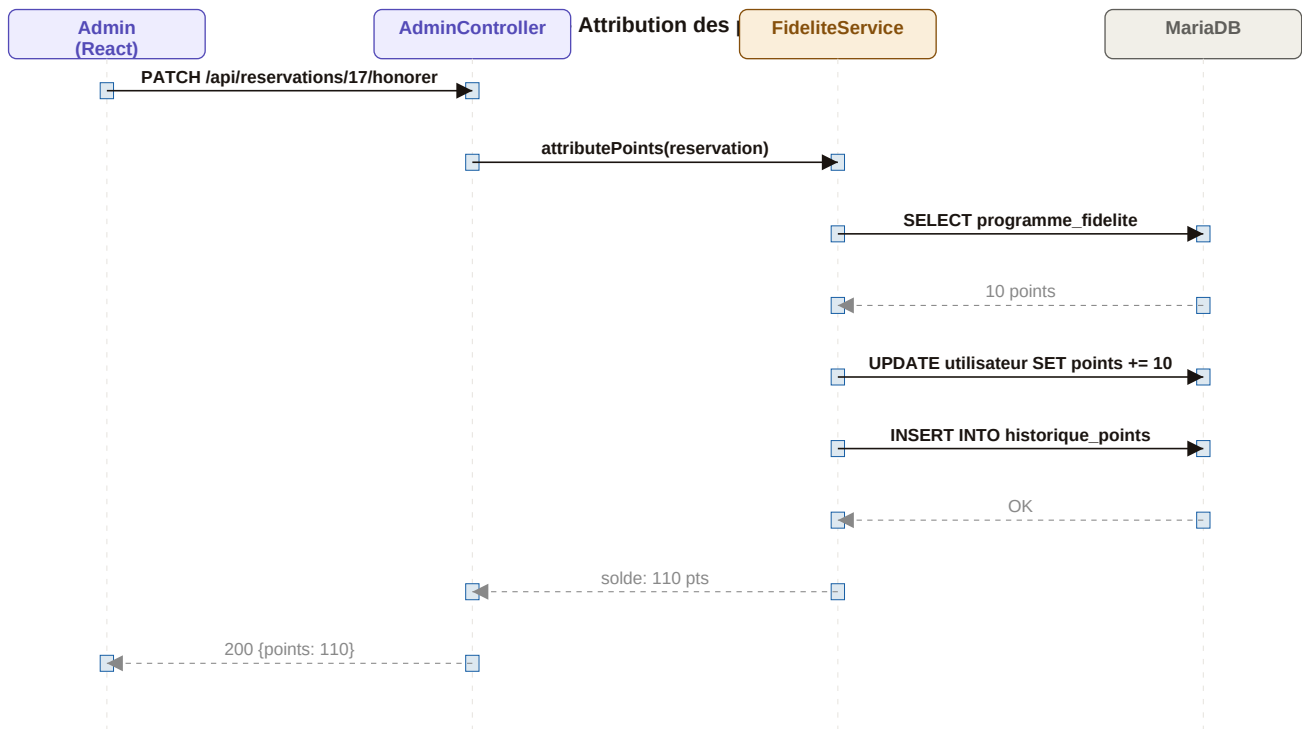
2.2 Création d'une réservation

Le client choisit date, créneau et couverts. ReservationService vérifie la disponibilité en base, crée la réservation puis déclenche l'envoi de l'email.



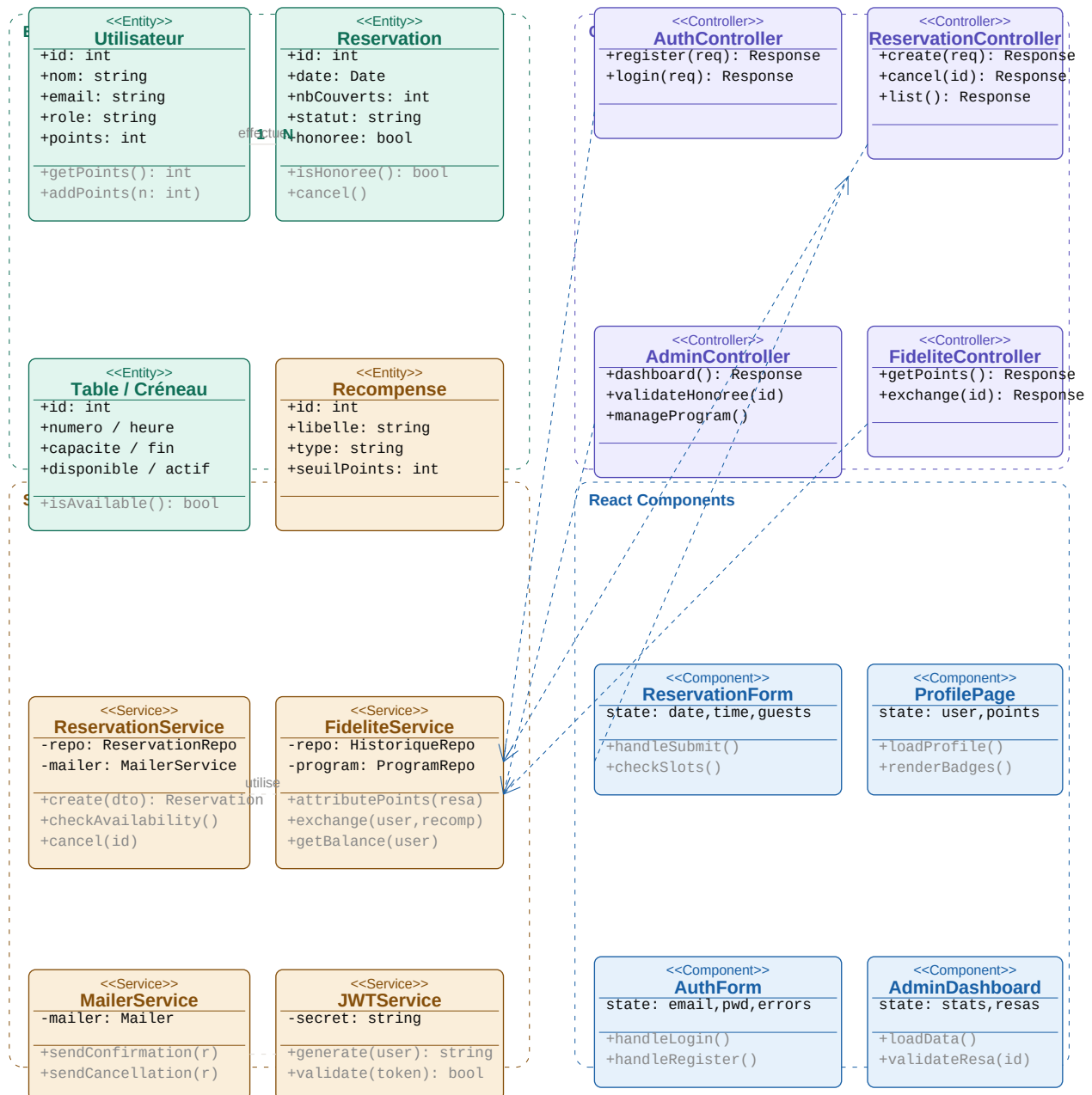
2.3 Attribution des points de fidélité

L'admin valide une réservation comme honorée. FideliteService récupère la configuration du programme, calcule les points, met à jour le solde et insère une ligne dans l'historique.



3. Diagramme de classes

Le diagramme est organisé en quatre packages. Les **Entities** correspondent aux entités Doctrine. Les **Controllers** reçoivent les requêtes HTTP et retournent du JSON. Les **Services** encapsulent la logique métier. Les **React Components** consomment l'API via HTTP.



Flèches pleines = associations (avec cardinalités sur les entités). Flèches en tirets bleus = dépendances d'usage (contrôlers → services, composants → API).

4. Architecture multi-couches

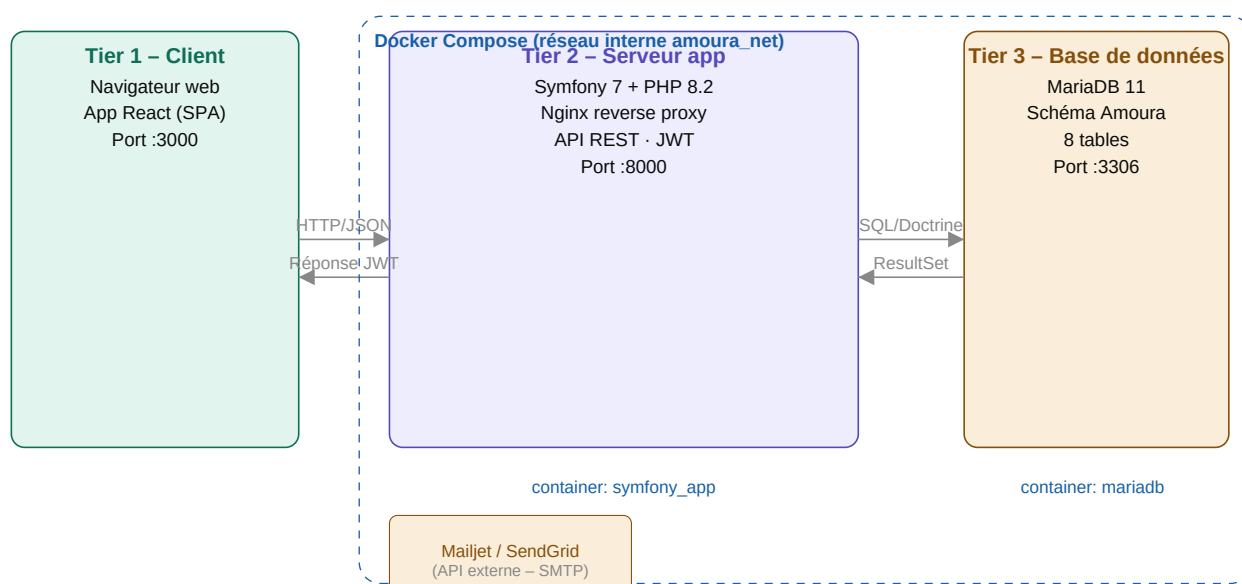
4.1 Pattern MVC adapté à l'API REST

J'ai choisi une architecture **API REST Symfony + SPA React**. C'est une variante du MVC où la Vue est entièrement côté client.

- **Modèle** : entités Doctrine (src/Entity/) + repositories (src/Repository/). Doctrine gère la persistance et les migrations.
- **Contrôleur** : classes dans src/Controller/ qui reçoivent les requêtes HTTP, valident les données et délèguent aux services. Ils retournent du JSON, jamais du HTML.
- **Vue** : application React qui consomme l'API. La séparation est nette : le back ne sait rien du front.
- **Services** (src/Service/) : couche métier indépendante — ReservationService, FideliteService, MailerService, JWTService.

4.2 Architecture n-tiers et Docker

L'application suit une architecture **3-tiers** : client, serveur applicatif et base de données. Ces trois tiers sont déployés dans des conteneurs Docker distincts orchestrés par docker-compose.yml.



Les packages Controllers, Services et Entities tournent tous sur le même conteneur Symfony — ce sont des couches logiques, pas des tiers physiques. En production, les conteneurs peuvent être répartis sur plusieurs hôtes sans toucher au code.

4.3 Bonnes pratiques

- **Single Responsibility** : chaque service a un rôle unique. ReservationService ne touche pas aux points fidélité.

- **DRY** : la vérification des disponibilités est centralisée dans `ReservationService::checkAvailability()`.
- **Variables d'environnement** : clés API Mailjet, DSN MariaDB, secret JWT — tout dans `.env.local`, jamais en dur.
- **PSR-12** : style de code PHP vérifié automatiquement par PHP CS Fixer dans la CI.
- **CORS** : `NelmioCorsBundle` autorise uniquement l'origine React (`localhost:3000` en dev, domaine de prod en production).

4.4 Bibliothèques et bundles

Composant	Rôle
<code>lexik/jwt-authentication-bundle</code>	Génération et validation des tokens JWT
<code>nelmio/cors-bundle</code>	En-têtes CORS pour la SPA React
<code>symfony/mailer</code> + Mailjet	Emails transactionnels (confirmation, annulation)
<code>symfony/validator</code>	Validation des données en entrée des endpoints API
<code>doctrine/orm</code>	ORM – mapping entités ↔ tables MariaDB
<code>axios</code> (npm)	Client HTTP React pour les appels API
<code>react-router-dom</code>	Routage côté client (SPA)
<code>zustand</code>	Gestion d'état global React (user, token)